



Publicación de notas:

24 de julio de 2024

Revisión:

26 de julio de 2024 de 10:30-12:30

FIRMA

Apellidos: _____

Nombre: _____

DNI: _____

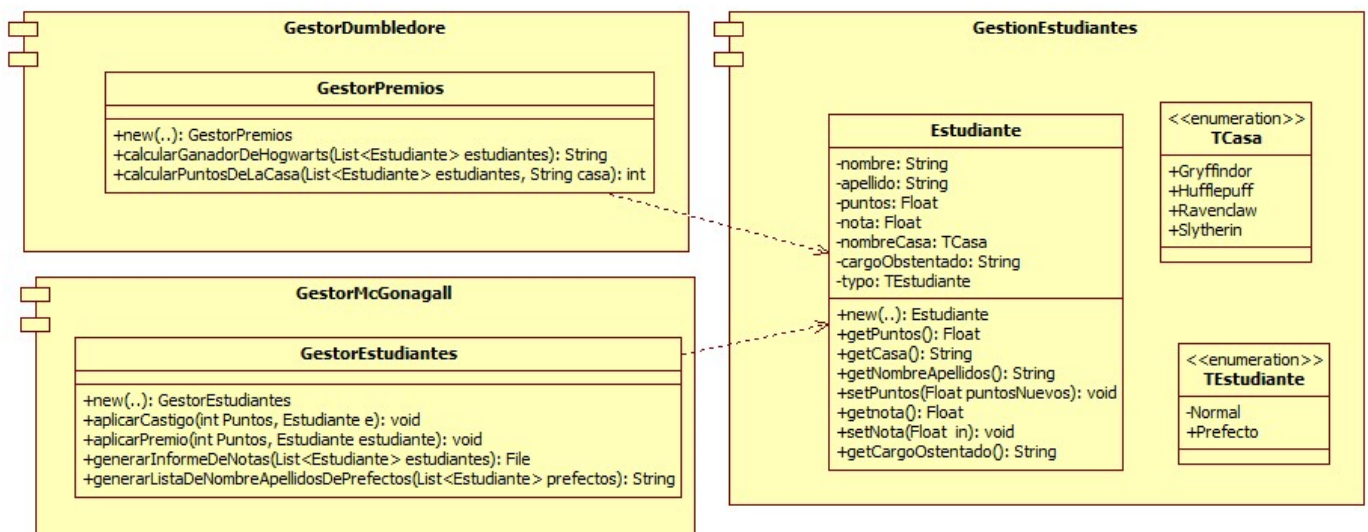
PROBLEMA 2 (5 puntos)

En el mundo de *Harry Potter* hay mucha magia, pero la escuela de hechicería de *Hogwarts* todavía sigue sin informatizarse y las actas de notas medias de alumnos y puntos de las casas se siguen escribiendo en papel y *vuelapluma*. El director *Dumbledore* ha decidido tomar el minotauro por los cuernos y digitalizar la escuela de inmediato. Aunque el dinero no es impedimento, sí le preocupa que el diseño sea de alta calidad por lo que ha seleccionado a los alumnos más prometedores de *FIS* para que resuelvan el problema de manera teórica.

El sistema deberá proveer el componente *GestorDumbledore* que necesita el director *Dumbledore* para poder otorgar los premios de la escuela basado en los puntos de los estudiantes y el componente *GestorMcGonagall* que necesita la profesora *McGonagall* para la generación de informes, listados de alumnos y aplicación de castigos y premios de puntos a los alumnos.

La escuela de *Hogwarts* está dividida en “casas” (Casa Gryffindor, Casa Hufflepuff, Casa Ravenclaw y Casa Slytherin) de tal manera todo estudiante pertenece a una (y solo una) de ellas. Los *estudiantes* tendrán, por tanto, una casa asignada, nombre y apellidos, una nota media en la escuela y unos puntos asociados obtenidos del año en curso. Los *prefectos* son un tipo especial de estudiantes que se encargan de mantener el orden en las “casas”. El sistema deberá almacenar el nombre del cargo que ostentan los prefectos y calcular sus puntos de forma diferente al del resto de estudiantes.

Tales requisitos han dado como resultado el diseño representado en la siguiente imagen.





Apartado 2.1 (2 puntos)

Dado el diagrama propuesto en el enunciado, asumiendo las necesidades explicadas y que:

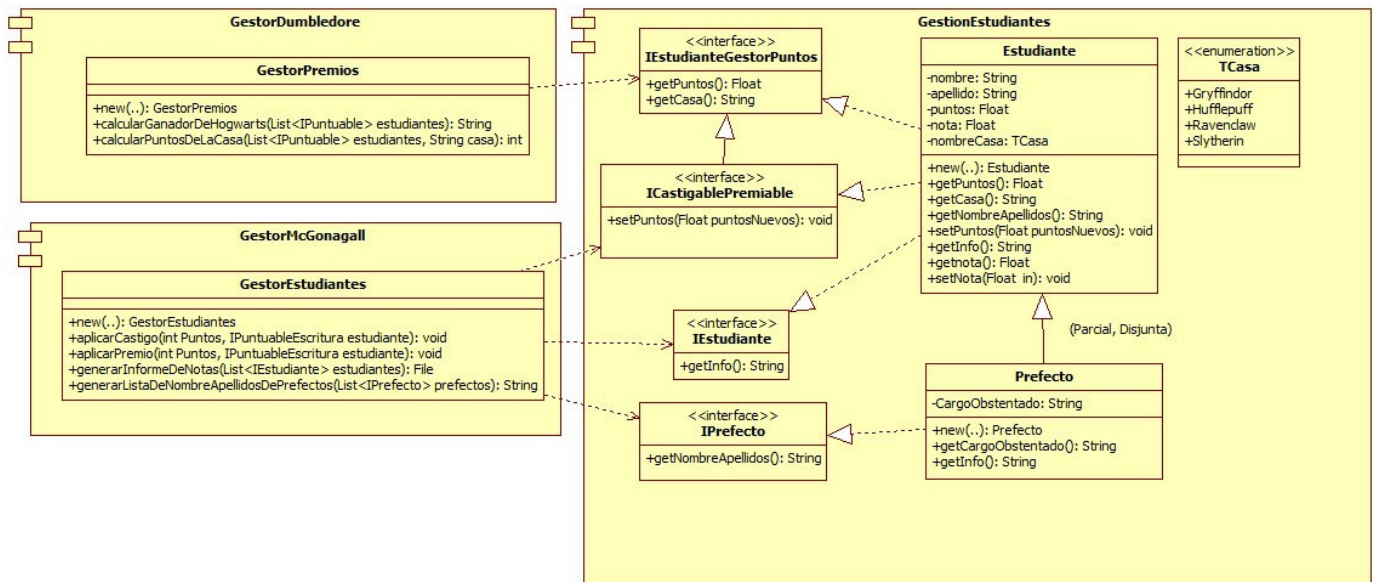
- **calcularGanadorDeHogwarts** y **calcularPuntosDeLaCasa** requerirán conocer la casa de cada estudiante y sus puntos.
- **aplicarCastigo** y **aplicarPremio** requerirá conocer y modificar los puntos del estudiante, así como conocer la casa a la que éste pertenece.
- **generarInformeDeNotas** requerirá obtener toda la información de los estudiantes incluyendo la información específica de los prefectos sin conocimiento explícito de su diferencia de clase.
- **generarListaDeNombreApellidosDePrefectos** requerirá obtener el nombre y el apellido juntos de cada prefecto.

Crea una nueva propuesta de diseño que **reduzca el acoplamiento** entre componentes. **Indica** que principios de diseño has aplicado en el diseño propuesto.

Principios de diseño aplicados:

SOLUCION

Principios de diseño aplicados: Sustitución de Liskov, Abierto/Cerrado, Segregación de Interfaces e Inversión de dependencias, abstracción por especialización, cohesión.





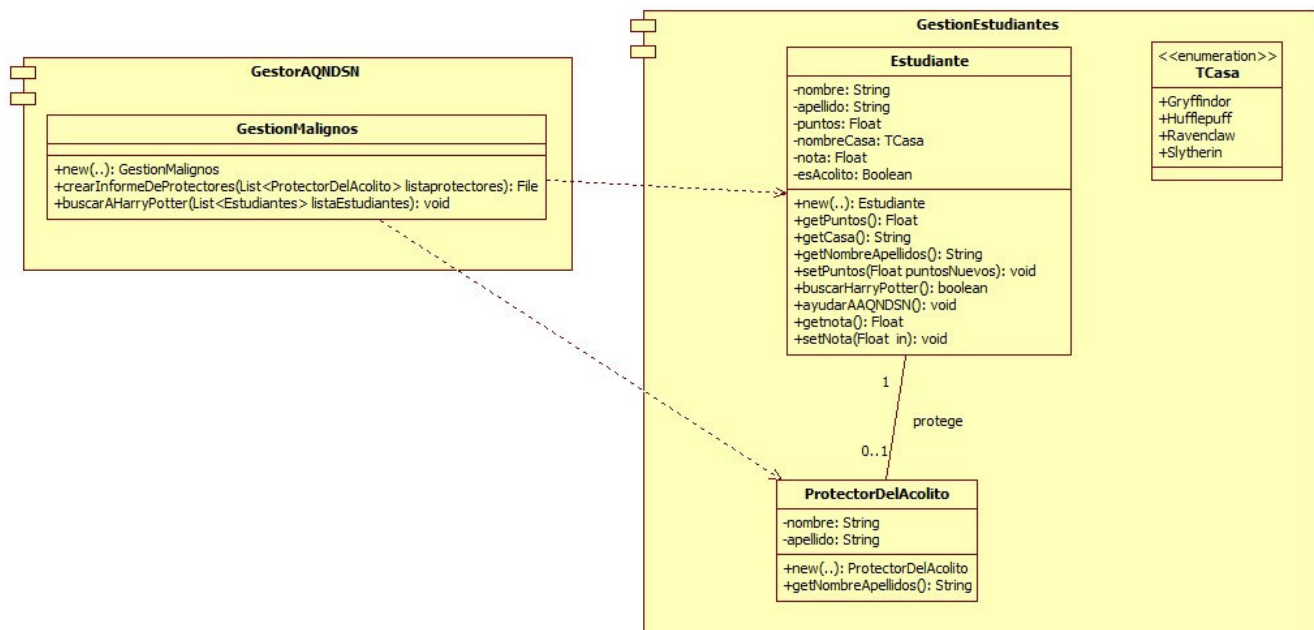
Apartado 2.2 (2 puntos)

Voldemort, “Aquel que no debe ser nombrado” o “AQNDSN” se ha enterado del plan de digitalización de la escuela de *Dumbledore* y ha pedido introducir cambios en el sistema para permitir tener controlados a sus *acólitos*, que son estudiantes a favor de su causa. Para ello, ha propuesto un diseño básico que resuelve sus necesidades.

En el diseño propuesto por AQNDSN, la clase *Estudiante* contiene nuevos atributos para reflejar la información de los *acólitos*. En este diseño también se ha incorporado a los *protectores*, que son personas a cargo de un acólito y que los protegen.

AQNDSN ha incluido en el diseño un componente *GestorAQNDSN* para poder operar con sus acólitos y con sus protectores de manera que pueda crear listados de estos y operaciones para buscar a *Harry Potter* y ayudarle con sus planes malvados.

A continuación se muestra el diagrama propuesto por AQNDSN:



Teniendo en cuenta las siguientes especificaciones:

- **crearInformeDeProtectores** solo necesita conocer el nombre y el apellido de manera conjunta por parte de los *Protectores*.
- **buscarAHarryPotter** ejecuta la operación *buscarHarryPotter* en cada *acólito*.

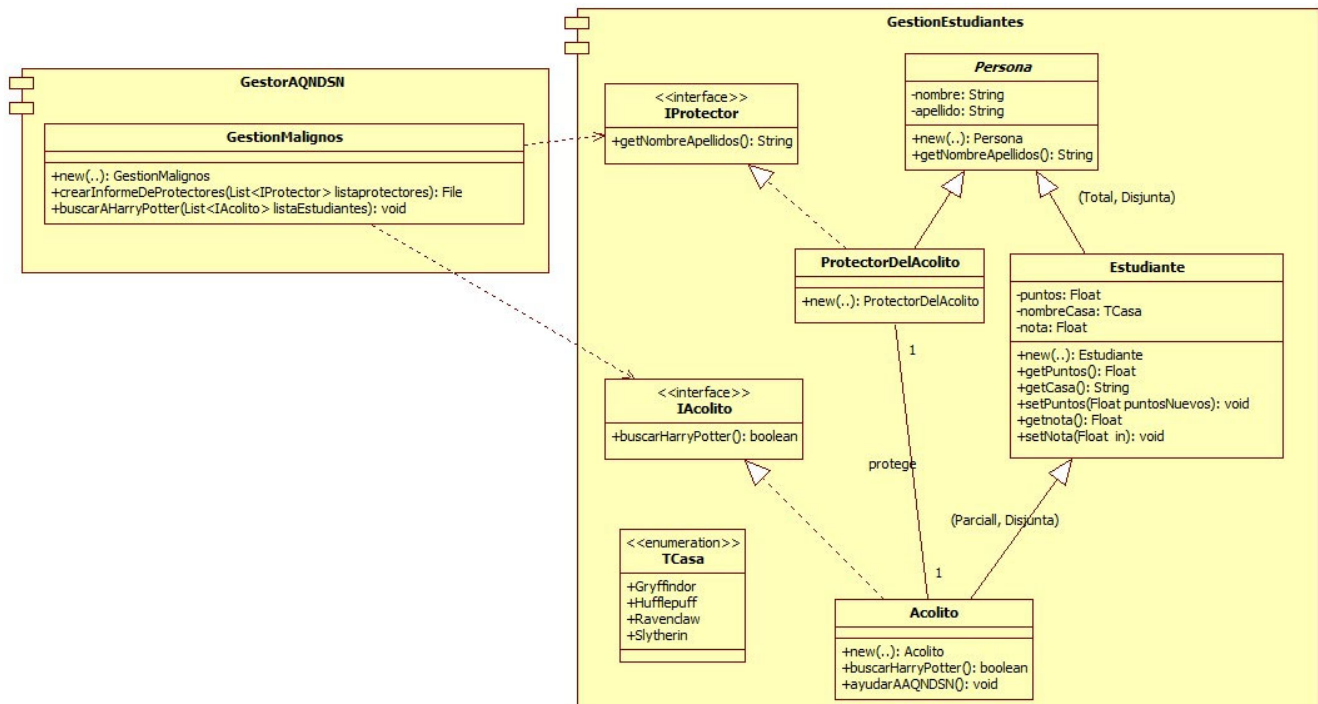
Crea una nueva propuesta de diseño que **reduzca el acoplamiento** entre componentes y mejore la cohesión de las clases. **Indica** que principios de diseño has aplicado en el diseño propuesto.



Principios de diseño aplicados:

SOLUCION

Principios de diseño aplicados: Abstracción por Generalización, Abstracción por Especialización, Inversión de Dependencias, Cohesión, Responsabilidad Única, Abierto/Cerrado



[illegible]



11 de julio de 2024

Apartado 2.3.2 (0.5 puntos)

Basado en la tabla anterior se pide rellenar la siguiente tabla de casos de prueba con la especificación de entrada y salida para la tabla de equivalencias creada en el apartado anterior y que sirva para la creación de las pruebas de Junit correspondientes a las pruebas de caja negra de la función *Combate*.

[illegible]

SOLUCION

Apartado 2.3.1 (0.5 puntos)

Variable	Clase Valida	Clase Invalida
CNombre1	V1."Hermion"	NI.""
CNombre2	V2."Ginny"	N2.""
Ganador	V3."C1" V4."C2"	N3."C3"
Codigo_Combate	V5."A"	N4."\$"

Apartado 2.3.2 (0.5 puntos)

Caso de Prueba	Entrada	Salida
CP0	{"Hermion", "Ginny", "C1", "A"}	True
CP1	{"Hermion", "Ginny", "C1", "\$"}	False
CP2	{"Hermion", "Ginny", "C2", "A"}	True
CP3	{"Hermion", "Ginny", "C3" V5	False
CP4	{"Hermion", "", "C1", "A"}	False
CP5	{"", "Ginny", "C1", "A"}	False